Lecture 3
Accessing Arrays using Pointers

Relationship Between Pointers and Arrays

- Arrays and pointers are closely related
- Arrays are laid out sequentially in memory
- By using the address-of operator (&), we can determine that arrays are laid out sequentially in memory. For example,

```
int array[5 ] = { 9, 7, 5, 3, 1 };
std::cout << "Element 0 is at address: " << &array[0] << '\n';
std::cout << "Element 1 is at address: " << &array[1] << '\n';
std::cout << "Element 2 is at address: " << &array[2] << '\n';
std::cout << "Element 3 is at address: " << &array[3] << '\n';
```

- **Each of these memory addresses is 4 bytes apart, which is the size of an integer**

Relationship Between Pointers and Arrays (Cont.)

- Accessing array elements with pointers
  - Assume declarations:

```
int List[ 5 ]; //declare array
int *LPtr;     //declare a pointer
LPtr = List;  // Address of first Element of array List
```

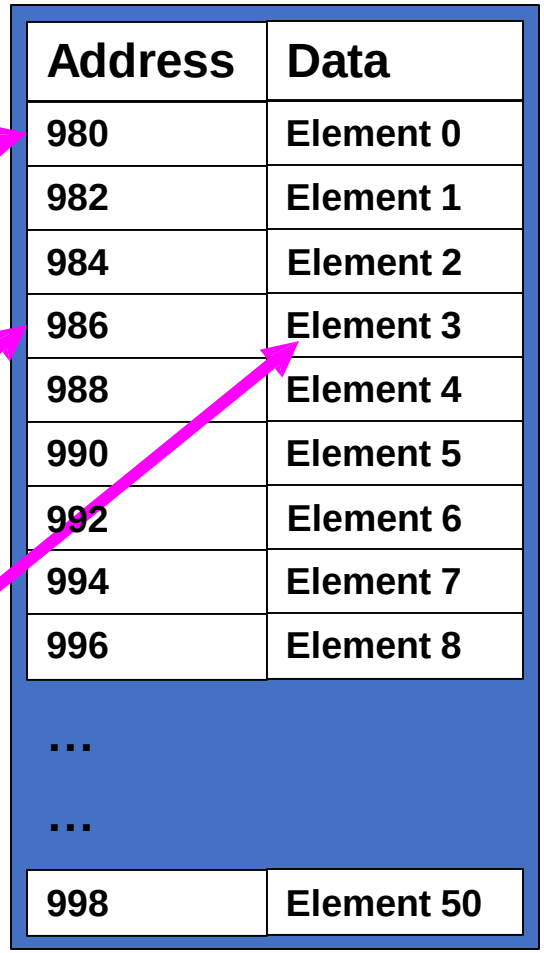
Effect:-

- List is an address, no need for &
- The LPtr pointer will contain the address of the first element of **array List**.

  - Element **List[2]** can be accessed by ***(LPtr + 2)**

Accessing 1-Demensional Array Using Pointers

- We know, Array name denotes the memory address of its first slot.
  - Example:
    - **int List [50];**
    - **int *Pointer;**
    - **Pointer = List; //sores address of the first element in Pointer**
- Other slots of the Array (List [50]) can be accessed using by performing Arithmetic operations on Pointer.
- For example the address of (element 4th) can be accessed using:-
  - int *Value = Pointer + 3;
- The value of (element 4th) can be accessed using:-
  - int Value = *(Pointer + 3);



Accessing 1-Demensional Array

```
....
....
      int List [ 50 ];
      int *Pointer;
      Pointer = List; // Address of first Element

      int *ptr;
cout<<*ptr; //what will it display?

 *ptr is equivalent to List[0]

ptr = Pointer + 3; // Address of 4th Element

      *ptr = 293; // 293 value store at 4th element
      address
```

Address	Data
980	Element 0
982	Element 1
984	Element 2
986	293
988	Element 4
990	Element 5
992	Element 6
994	Element 7
996	Element 8
...	
...	
998	Element 50

Accessing 1-Demensional Array

We can access all element of List [50] using Pointers and for loop combinations.

```
...
...
    int List [ 50 ];
    int *Pointer;
    Pointer = List;
    for ( int i = 0; i < 50; i++ )
    {
        cout << *Pointer;
        Pointer++; // Address of next element
    }
```

Address	Data
980	Element 0
982	Element 1
984	Element 2
986	Element 3
988	Element 4
990	Element 5
992	Element 6
994	Element 7
996	Element 8
...	
...	
998	Element 50

This is Equivalent to

```
for ( int loop = 0; loop < 50; loop++ )
    cout << Array [ loop ] ;
```